SSD_FLASH write protection setting guide

---

1. Use the background

In actual project use, when the Kernel is started, many partitions are only read-only partitions for the Kernel. However, these partitions can be modified by mtd devices or abnormally modified due to some uncertain factors. Often such modifications will cause the system to fail to start. Therefore, for such scenarios, the user interface of Flash Write Protect has been added.

---

2. How to use

1. Kernel Config does not turn on the Write Protect function by default, you need to turn it on manually:

   NOR: CONFIG_SS_FLASH_ISP_WP

   NAND: CONFIG_SS_SPINAND_WP

   Note: Only one of the two Configs can be opened at the same time.

2. The operation of Flash Write Protect is realized by reading and writing the protect and protect_valid files under /sys/class/mstar/msys/:

Get the current Flash protectable interval information:

```
/sys/devices/virtual/mstar/msys # cat protect_valid
Flash Valid Protect Area:
  0 : 0x00000000 - 0x00000000
  1 : 0x07FC0000 - 0x07FFFFFF
  2 : 0x07F80000 - 0x07FFFFFF
  3 : 0x07F00000 - 0x07FFFFFF
  4 : 0x07E00000 - 0x07FFFFFF
  5 : 0x07C00000 - 0x07FFFFFF
  6 : 0x07800000 - 0x07FFFFFF
  7 : 0x07000000 - 0x07FFFFFF
  8 : 0x06000000 - 0x07FFFFFF
  9 : 0x04000000 - 0x07FFFFFF
 10 : 0x00000000 - 0x0003FFFF
 11 : 0x00000000 - 0x0007FFFF
 12 : 0x00000000 - 0x000FFFFF
 13 : 0x00000000 - 0x001FFFFF
 14 : 0x00000000 - 0x003FFFFF
 15 : 0x00000000 - 0x007FFFFF
 16 : 0x00000000 - 0x00FFFFFF
 17 : 0x00000000 - 0x01FFFFFF
 18 : 0x00000000 - 0x03FFFFFF
 19 : 0x00000000 - 0x07FFFFFF
/sys/devices/virtual/mstar/msys #
```

To set the current protected area:

```
/sys/devices/virtual/mstar/msys # echo 0x00000000 0x00000000 > protect
/sys/devices/virtual/mstar/msys #
```

Get the current protection area:

```
/sys/devices/virtual/mstar/msys # cat protect
Flash Protect Area:
 0x00000000 - 0x00000000
/sys/devices/virtual/mstar/msys #
```

---

3. Preparation Guidelines

3.1. Principle description

Flash 的 Write Protect 功能一般通过设置 Flash的某些寄存器实现，具体的操作方式可以参考 Flash 的 Datasheet。各厂商的 Flash支持的 Write Protect 功能都有一些差异，但基本的保护功能大同小异。

如 W25Q128的 Write Protect 功能是通过写 Status Register 1 的 bit 6 : bit2 实现：

7.1 Status Registers

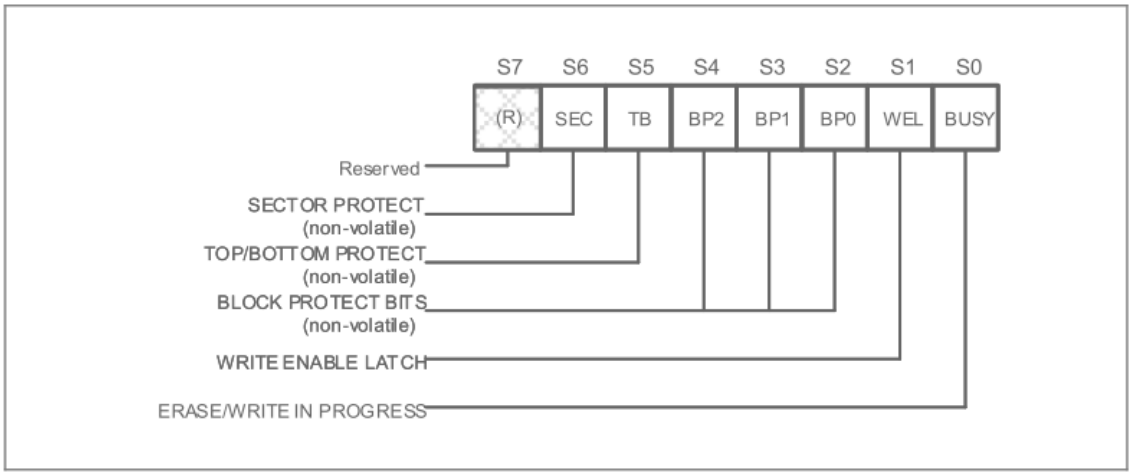

Figure 4a. Status Register-1

读写 Status Register 1 的方法：

8.2.5 Write Status Register-1 (01h), Status Register-2 (31h) & Status Register-3 (11h)

各bit值对应的保护区域为：

7.1.14 W25Q128JV Status Register Memory Protection (WPS = 0, CMP = 0)

STATUS REGISTER(1)					W25Q128JV (128M-BIT) MEMORY PROTECTION(3)			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION(2)
X	X	0	0	0	NONE	NONE	NONE	NONE
0	0	0	0	1	252 thru 255	FC0000h – FFFFFFh	256KB	Upper 1/64
0	0	0	1	0	248 thru 255	F80000h – FFFFFFh	512KB	Upper 1/32
0	0	0	1	1	240 thru 255	F00000h – FFFFFFh	1MB	Upper 1/16
0	0	1	0	0	224 thru 255	E00000h – FFFFFFh	2MB	Upper 1/8
0	0	1	0	1	192 thru 255	C00000h – FFFFFFh	4MB	Upper 1/4
0	0	1	1	0	128 thru 255	800000h – FFFFFFh	8MB	Upper 1/2
0	1	0	0	1	0 thru 3	000000h – 03FFFFh	256KB	Lower 1/64
0	1	0	1	0	0 thru 7	000000h – 07FFFFh	512KB	Lower 1/32
0	1	0	1	1	0 thru 15	000000h – 0FFFFFh	1MB	Lower 1/16
0	1	1	0	0	0 thru 31	000000h – 1FFFFFh	2MB	Lower 1/8
0	1	1	0	1	0 thru 63	000000h – 3FFFFFh	4MB	Lower 1/4
0	1	1	1	0	0 thru 127	000000h – 7FFFFFh	8MB	Lower 1/2
X	X	1	1	1	0 thru 255	000000h – FFFFFFh	16MB	ALL
1	0	0	0	1	255	FFF000h – FFFFFFh	4KB	U - 1/4096
1	0	0	1	0	255	FFE000h – FFFFFFh	8KB	U - 1/2048
1	0	0	1	1	255	FFC000h – FFFFFFh	16KB	U - 1/1024
1	0	1	0	X	255	FF8000h – FFFFFFh	32KB	U - 1/512
1	1	0	0	1	0	000000h – 000FFFh	4KB	L - 1/4096
1	1	0	1	0	0	000000h – 001FFFh	8KB	L - 1/2048
1	1	0	1	1	0	000000h – 003FFFh	16KB	L - 1/1024
1	1	1	0	X	0	000000h – 007FFFh	32KB	L - 1/512

Notes:
1. X = don't care
2. L = Lower; U = Upper
3. If any Erase or Program command specifies a memory region that contains protected data portion, this command will be ignored.

如上图， WPS 为是否使用每个 Block 单独设置 Write Protect 功能，该功能不是所有 Flash 厂商都支持，所以我们的驱动暂时没有支持 WPS = 1 的使用场景。

图中 CMP 为是否支持 Write Protest 功能范围细化，该功能也并不是所有 Flash 厂商都支持，所以我们的驱动也暂时没有支持 CMP = 1 的使用场景。

SPINAND 的原理和 SPINOR 是相同的，仅差 SPINAND 和 SPINOR 读写的寄存器不一样以及发送的命令不一样，如 W25N01G：

7.1 Protection Register / Status Register-1 (Volatile Writable, OTP lockable)

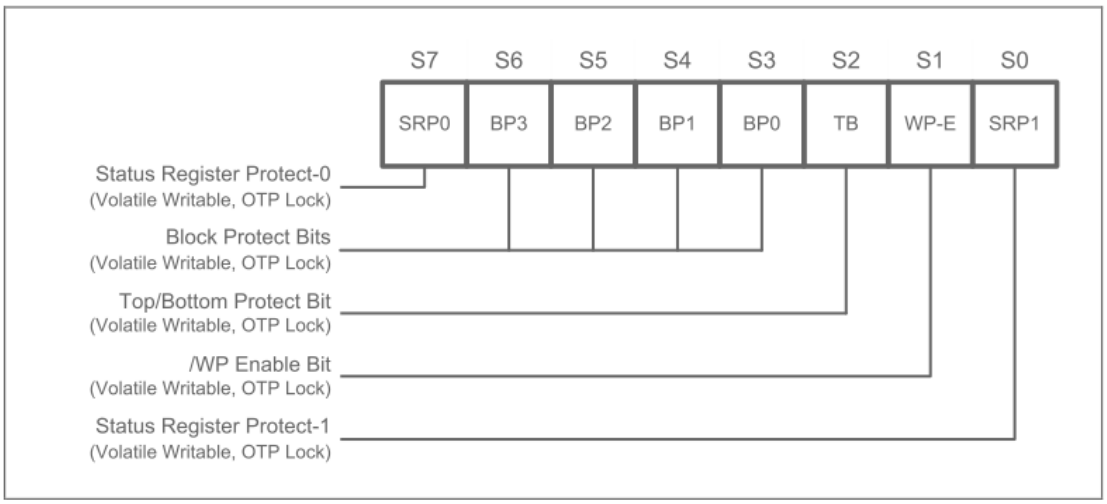

Figure 4a. Protection Register / Status Register-1 (Address Axh)

读写 Status Register 1 的方法：

8.2.3 Read Status Register (0Fh / 05h)

8.2.4 Write Status Register (1Fh / 01h)

各bit值对应的保护区域为：

7.4 W25N01GV Status Register Memory Protection

STATUS REGISTER(1)					W25N01GV (1G-BIT / 128M-BYTE) MEMORY PROTECTION(2)			
TB	BP3	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED PAGE ADDRESS PA[15:0]	PROTECTED DENSITY	PROTECTED PORTION
X	0	0	0	0	NONE	NONE	NONE	NONE
0	0	0	0	1	1022 & 1023	FF80h - FFFFh	256KB	Upper 1/512
0	0	0	1	0	1020 thru 1023	FF00h - FFFFh	512KB	Upper 1/256
0	0	0	1	1	1016 thru 1023	FE00h - FFFFh	1MB	Upper 1/128
0	0	1	0	0	1008 thru 1023	FC00h - FFFFh	2MB	Upper 1/64
0	0	1	0	1	992 thru 1023	F800h - FFFFh	4MB	Upper 1/32
0	0	1	1	0	960 thru 1023	F000h - FFFFh	8MB	Upper 1/16
0	0	1	1	1	896 thru 1023	E000h - FFFFh	16MB	Upper 1/8
0	1	0	0	0	768 thru 1023	C000h - FFFFh	32MB	Upper 1/4
0	1	0	0	1	512 thru 1023	8000h - FFFFh	64MB	Upper 1/2
1	0	0	0	1	0 & 1	0000h – 007Fh	256KB	Lower 1/512
1	0	0	1	0	0 thru 3	0000h - 00FFh	512KB	Lower 1/256
1	0	0	1	1	0 thru 7	0000h - 01FFh	1MB	Lower 1/128
1	0	1	0	0	0 thru 15	0000h - 03FFh	2MB	Lower 1/64
1	0	1	0	1	0 thru 31	0000h - 07FFh	4MB	Lower 1/32
1	0	1	1	0	0 thru 63	0000h - 0FFFh	8MB	Lower 1/16
1	0	1	1	1	0 thru 127	0000h - 1FFFh	16MB	Lower 1/8
1	1	0	0	0	0 thru 255	0000h - 3FFFh	32MB	Lower 1/4
1	1	0	0	1	0 thru 511	0000h - 7FFFh	64MB	Lower 1/2
X	1	0	1	X	0 thru 1023	0000h - FFFFh	128MB	ALL
X	1	1	X	X	0 thru 1023	0000h - FFFFh	128MB	ALL

3.2. 增加驱动支持

SPINOR Flash 需要增加 `drivers/sstar/flash_isp/drvDeviceInfo.c` 中 `_hal_SERFLASH_table` 数组的 pWriteProtectTable 成员，该成员为 Write Protect 的保护信息：

```
{ FLASH_IC_W25Q128,     MID_WB,     0x40,   0x18,   _pstWriteProtectTable_W25Q128,
```

_pstWriteProtectTable_W25Q128 则为储存 W25Q128 Flash Write Protect 的信息：

```
ST_WRITE_PROTECT _pstWriteProtectTable_W25Q128[] =
{
    //   BPX,                    Lower Bound         Upper Bound
    {   BITS(6:2, 0),   BMASK(6:2),   0x00000000,     0x00000000  },
    {   BITS(6:2, 1),   BMASK(6:2),   0x00FC0000,     0x00FFFFFF  },
    {   BITS(6:2, 2),   BMASK(6:2),   0x00F80000,     0x00FFFFFF  },
    {   BITS(6:2, 3),   BMASK(6:2),   0x00F00000,     0x00FFFFFF  },
    {   BITS(6:2, 4),   BMASK(6:2),   0x00E00000,     0x00FFFFFF  },
    {   BITS(6:2, 5),   BMASK(6:2),   0x00C00000,     0x00FFFFFF  },
    {   BITS(6:2, 6),   BMASK(6:2),   0x00800000,     0x007FFFFF  },
    {   BITS(6:2, 9),   BMASK(6:2),   0x00000000,     0x0003FFFF  },
    {   BITS(6:2, 10),  BMASK(6:2),   0x00000000,     0x0007FFFF  },
    {   BITS(6:2, 11),  BMASK(6:2),   0x00000000,     0x000FFFFF  },
    {   BITS(6:2, 12),  BMASK(6:2),   0x00000000,     0x001FFFFF  },
    {   BITS(6:2, 13),  BMASK(6:2),   0x00000000,     0x003FFFFF  },
    {   BITS(6:2, 14),  BMASK(6:2),   0x00000000,     0x007FFFFF  },
    {   BITS(6:2, 15),  BMASK(6:2),   0x00000000,     0x00FFFFFF  },
    {   BITS(6:2, 0),   BMASK(6:2),   0xFFFFFFFF,     0xFFFFFFFF  },
};
```

数组的第一列为 Status Register 1 的值， 第二列为值的Mask，第三列为保护范围的低地址，第四列为保护范围的高地址。

SPINAND Flash 需要增加 `drivers/sstar/spinand/drv/mdrv_spinand_dev.c` 中 `_gtSpinandWpTable` 的 pWriteProtectTable 成员，该成员为 Write Protect 的保护信息：

```
SPI_NAND_DEVICE_t _gtSpinandWpTable[] =
{
    {{0xEF, 0xAA, 0x21}, _pstWriteProtectTable_W25N01GV},

    {{0x00, 0x00, 0x00}, NULL},
};
```

_pstWriteProtectTable_W25N01GV stores W25N01GV Flash Write Protect information:

```
SPI_NAND_WP_t _pstWriteProtectTable_W25N01GV[] =
{
    //      BPX             Mask        Lower Bound     Upper Bound
    {   BITS(6:2,  0),   BMASK(6:2),   0x00000000,     0x00000000  },
    {   BITS(6:2,  2),   BMASK(6:2),   0x07FC0000,     0x07FFFFFF  },
    {   BITS(6:2,  4),   BMASK(6:2),   0x07F80000,     0x07FFFFFF  },
    {   BITS(6:2,  6),   BMASK(6:2),   0x07F00000,     0x07FFFFFF  },
    {   BITS(6:2,  8),   BMASK(6:2),   0x07E00000,     0x07FFFFFF  },
    {   BITS(6:2, 10),   BMASK(6:2),   0x07C00000,     0x07FFFFFF  },
    {   BITS(6:2, 12),   BMASK(6:2),   0x07800000,     0x07FFFFFF  },
    {   BITS(6:2, 14),   BMASK(6:2),   0x07000000,     0x07FFFFFF  },
    {   BITS(6:2, 16),   BMASK(6:2),   0x06000000,     0x07FFFFFF  },
    {   BITS(6:2, 18),   BMASK(6:2),   0x04000000,     0x07FFFFFF  },
    {   BITS(6:2,  3),   BMASK(6:2),   0x00000000,     0x0003FFFF  },
    {   BITS(6:2,  5),   BMASK(6:2),   0x00000000,     0x0007FFFF  },
    {   BITS(6:2,  7),   BMASK(6:2),   0x00000000,     0x000FFFFF  },
    {   BITS(6:2,  9),   BMASK(6:2),   0x00000000,     0x001FFFFF  },
    {   BITS(6:2, 11),   BMASK(6:2),   0x00000000,     0x003FFFFF  },
    {   BITS(6:2, 13),   BMASK(6:2),   0x00000000,     0x007FFFFF  },
    {   BITS(6:2, 15),   BMASK(6:2),   0x00000000,     0x00FFFFFF  },
    {   BITS(6:2, 17),   BMASK(6:2),   0x00000000,     0x01FFFFFF  },
    {   BITS(6:2, 19),   BMASK(6:2),   0x00000000,     0x03FFFFFF  },
    {   BITS(6:2, 21),   BMASK(6:2),   0x00000000,     0x07FFFFFF  },
    {   BITS(6:2,  0),   BMASK(6:2),   0xFFFFFFFF,     0xFFFFFFFF  },
};
```

Note: The last item u32LowerBound and u32UpperBound of the protect table must be 0xFFFFFFFF, which is the end of the array index.

The last item of the _hal_SERFLASH_table and _gtSpinandWpTable arrays is also a sign of the end of the array index, please do not modify it.

3.3. Set boot protection area

Because it is difficult for the driver to know how the user application wants to set the protection area, the default protection area is set in the compilation configuration of Alkaid, and it is more reasonable to set different protection areas according to different partitions.

Such `project/image/configs/i2m/spinand.ubifs.p2.partition.wp.config` as adding `FLASH_WP_RANGE` variables in :

```
IMAGE_LIST = cis ipl ipl_cust uboot logo kernel rootfs miservice customer appconfigs
OTA_IMAGE_LIST = ipl ipl_cust uboot logo kernel miservice customer appconfigs
FLASH_TYPE = spinand
UBI_MLC_TYPE = 0
PAT_TABLE = ubi
PHY_TEST = no
#overwrite CIS(BL0,BL1,UBOOT) PBAs
CIS_PBAs = 10 0 0
CIS_COPIES = 5
FLASH_WP_RANGE = 0x00000000 0x00FFFFFF
USR_MOUNT_BLOCKS:=miservice customer appconfigs
ENV_CFG = /dev/mtdl0 0x00000 0x1000 0x20000 2
ENV_CFG1 = /dev/mtdl1 0x00000 0x1000 0x20000 2
```